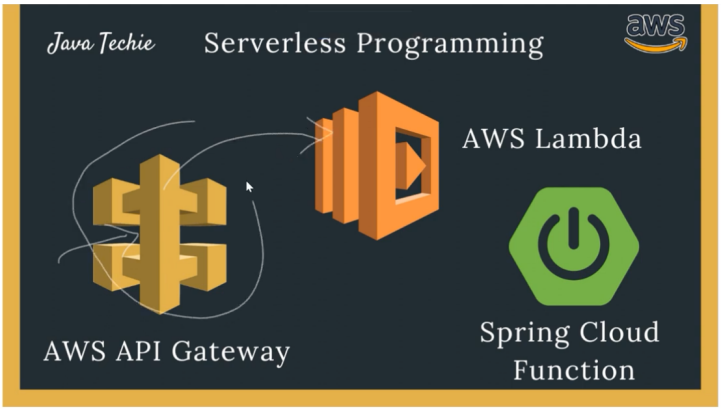
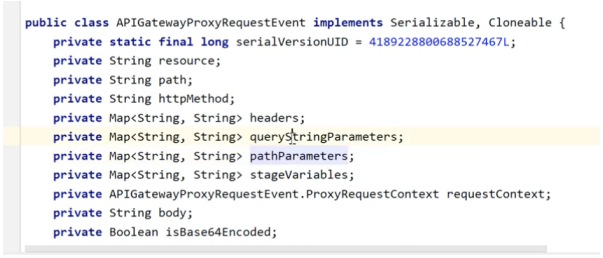
Trigger AWS Lambda using AWS API gateway request & API Gateway for spring boot application



We have existing lambda functions - https://github.com/Java-Techie-jt/aws-lambda-api-gateway

```
import ...

@SpringBootApplication
public class SpringbootAwsLambdaApplication {

    @Autowired
    private OrderDao orderDao;

    @Bean
    public Supplier<List<Order>> orders() {
        return () -> orderDao.buildOrders();
    }

    @Bean
    public Function<String, List<Order>> findOrderByName() {
        return (input) -> orderDao.buildOrders().stream().filter(order -> order.getNa
    }
```

In this Function Lambda we must use APIGatewayProxyRequestEvent – it has multiple way to pass the request



```
public class APIGatewayProxyRequestEvent implements Serializable, Cloneable {
    private static final long serialVersionUID = 418922880068852746 7L;
    private String resource;
    private String path;
    private String httpMethod;
    private Map<String, String> headers;
    private Map<String, String> queryStringParameters;
    private Map<String, String> pathParameters;
    private Map<String, String> stageVariables;
    private APIGatewayProxyRequestEvent.ProxyRequestContext requestContext;
    private String body;
    private Boolean isBase64Encoded;
```

aws-lambda-api-gateway / src / main / java / com / javatechie / aws / lambda / **OrderHandler.java**

basahota #code

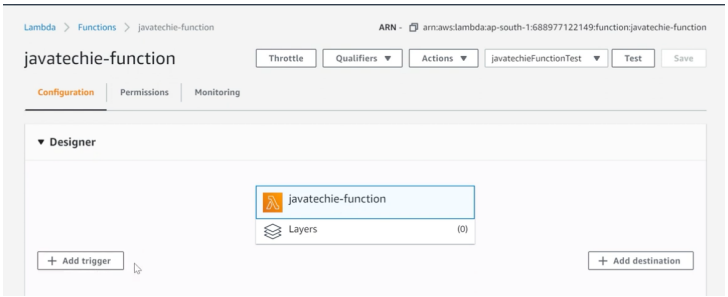
Code | Blame | 10 lines (7 loc) · 384 Bytes | Raw

```java
package com.javatechie.aws.lambda;

import com.amazonaws.services.lambda.runtime.events.APIGatewayProxyRequestEvent;
import com.javatechie.aws.lambda.domain.Order;
import org.springframework.cloud.function.adapter.aws.SpringBootRequestHandler;

import java.util.List;

public class OrderHandler extends SpringBootRequestHandler<APIGatewayProxyRequestEvent, List<Order>> {
}
```

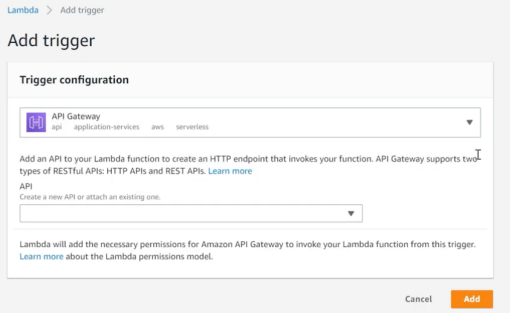
```java

@SpringBootApplication
public class SpringbootAwsLambdaApplication {

    @Autowired
    private OrderDao orderDao;

    @Bean
    public Supplier<List<Order>> orders() {
        return () -> orderDao.buildOrders();
    }

    @Bean
    public Function<APIGatewayProxyRequestEvent, List<Order>> findOrderByName() {
        return (requestEvent) -> orderDao.buildOrders()
                .stream()
                .filter(order -> order.getName().equals(requestEvent.getQueryStringParameters().get("orderName")))
                .collect(Collectors.toList());
    }

```

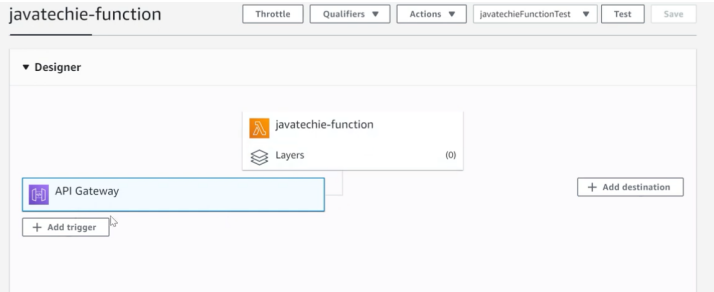
Build jar

Click on add trigger

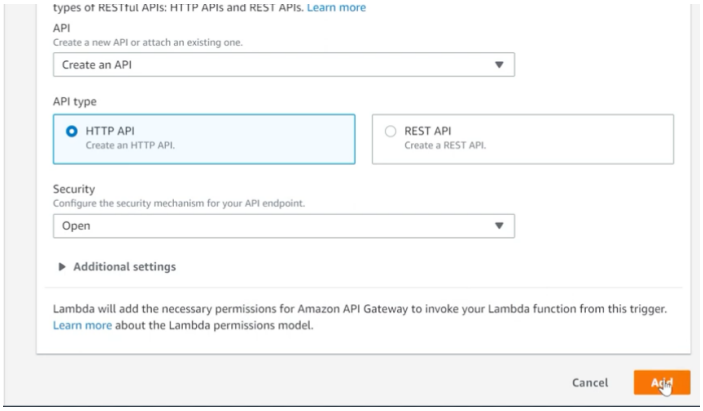


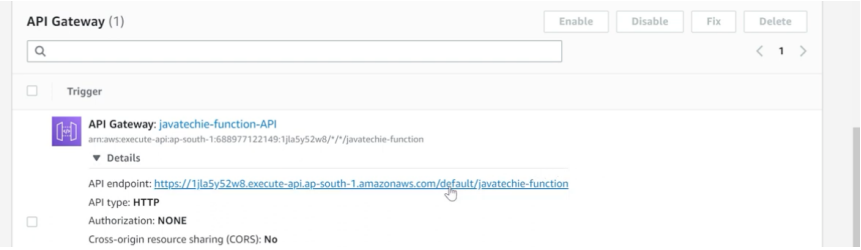
Click on the API gateway



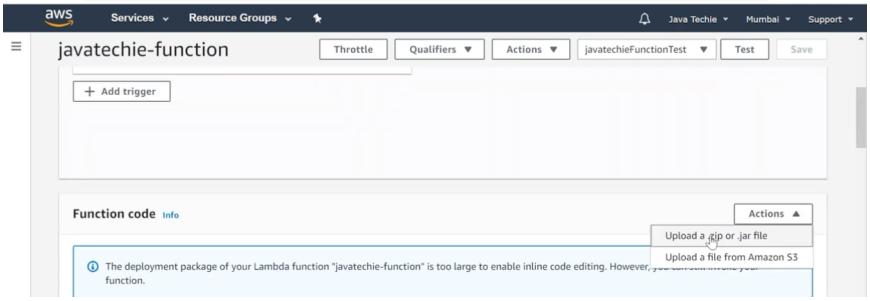


At the bottom we can see - API endpoint

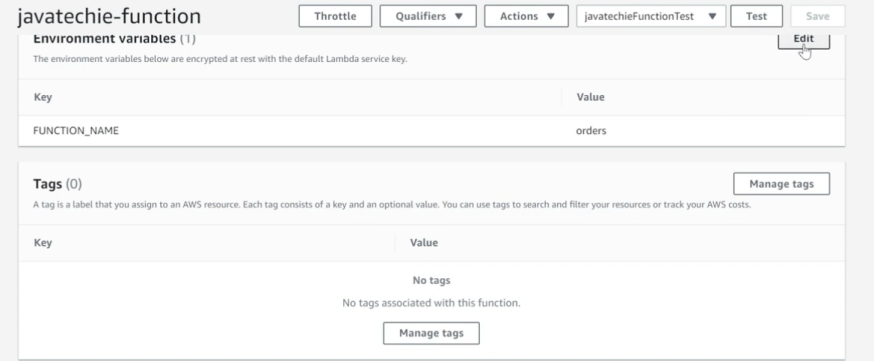


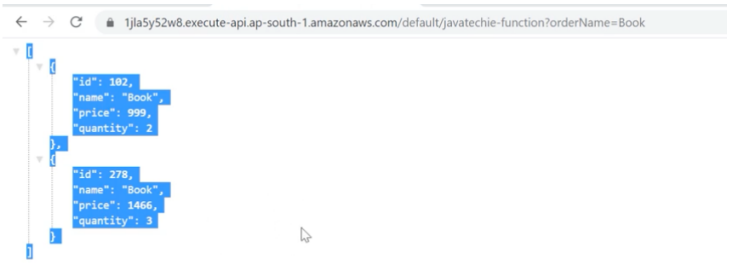


Add updated jar file

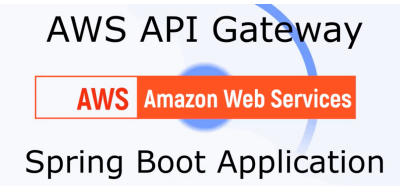


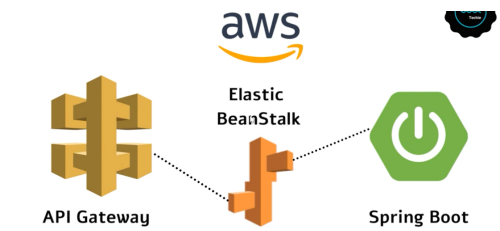
End point is controlled here



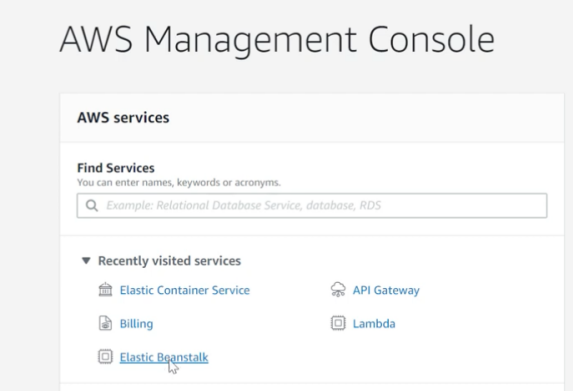


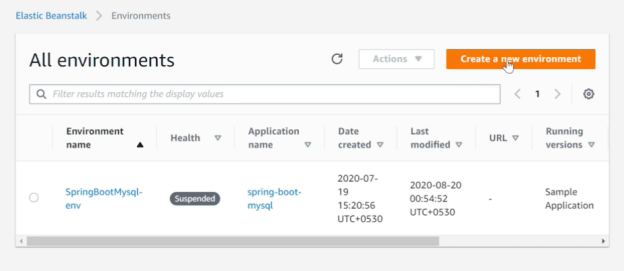
API Gateway – Spring boot Application.

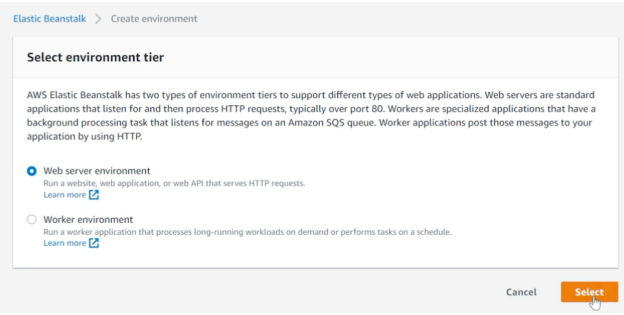


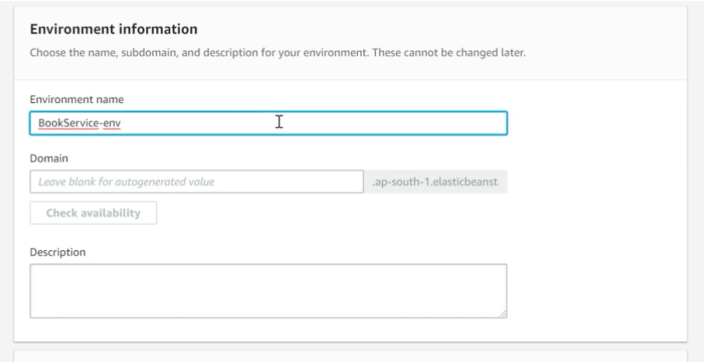


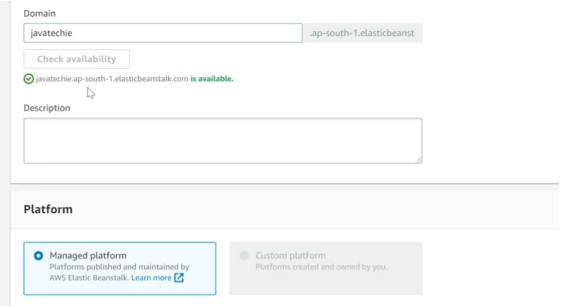
We call spring boot application deployed in elastic beanstalk via API gateway.

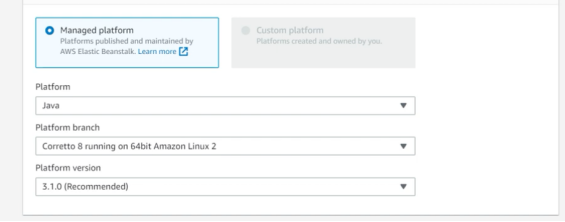




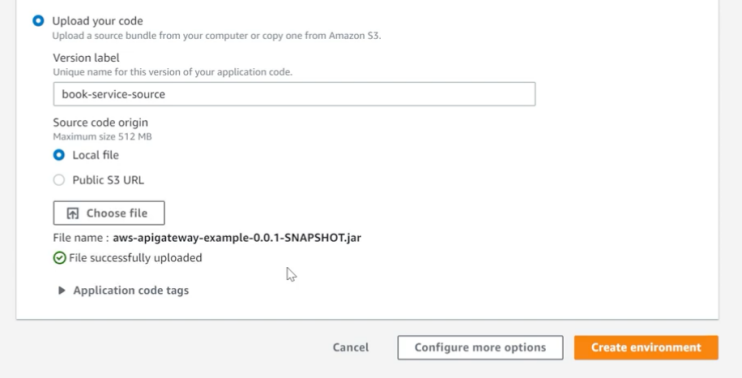


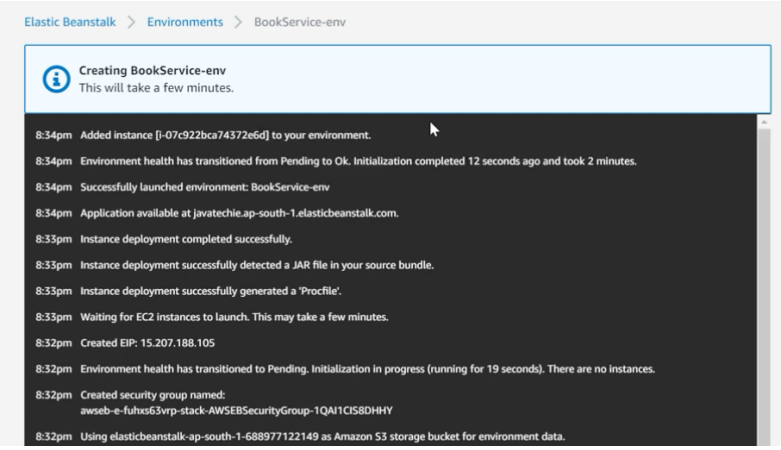


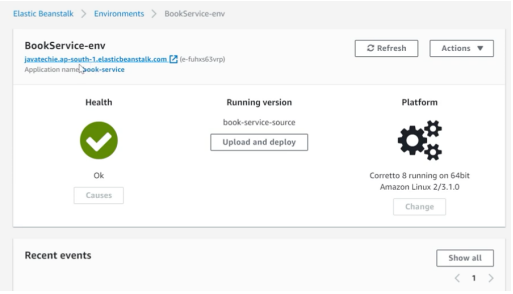


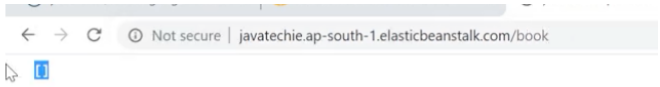


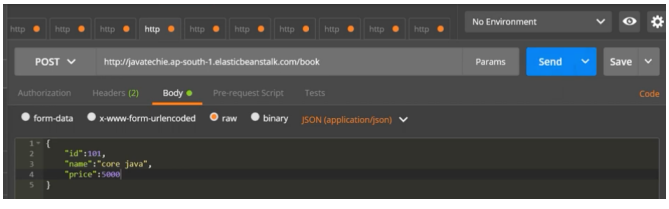
Upload your jar and create environment

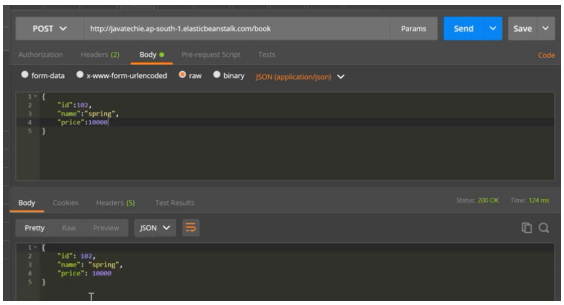

Upload your code
Upload a source bundle from your computer or copy one from Amazon S3.
Version label
Unique name for this version of your application code.
book-service-source
Source code origin
Maximum size 512 MB
Local file
Public S3 URL
Choose file
File name : aws-apigateway-example-0.0.1-SNAPSHOT.jar
File successfully uploaded
Application code tags
Cancel
Configure more options
Create environment


Elastic Beanstalk > Environments > BookService-env
Creating BookService-env
This will take a few minutes.
8:34pm Added instance [i-07c922bca74372e6d] to your environment.
8:34pm Environment health has transitioned from Pending to Ok. Initialization completed 12 seconds ago and took 2 minutes.
8:34pm Successfully launched environment: BookService-env
8:34pm Application available at javatechie.ap-south-1.elasticbeanstalk.com.
8:33pm Instance deployment completed successfully.
8:33pm Instance deployment successfully detected a JAR file in your source bundle.
8:33pm Instance deployment successfully generated a 'Procfile'.
8:33pm Waiting for EC2 instances to launch. This may take a few minutes.
8:32pm Created EIP: 15.207.188.105
8:32pm Environment health has transitioned to Pending. Initialization in progress (running for 19 seconds). There are no instances.
8:32pm Created security group named:
awseb-e-fuhxs63vrp-stack-AWSEBSecurityGroup-1QAI1CIS8DHHY
8:32pm Using elasticbeanstalk-ap-south-1-688977122149 as Amazon S3 storage bucket for environment data.

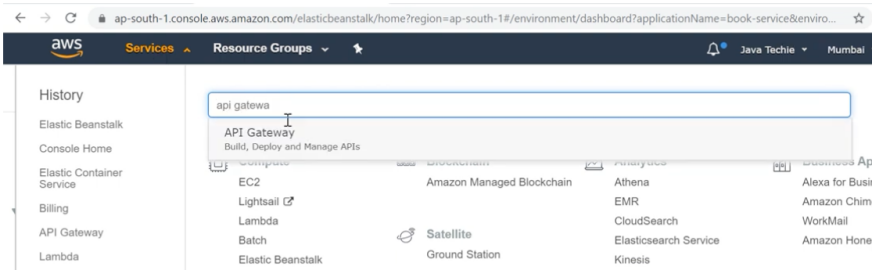

Elastic Beanstalk > Environments > BookService-env
BookService-env
Refresh
Actions
Health
Running version
Platform
book-service-source
Upload and deploy
Ok
Causes
Corretto 8 running on 64bit
Amazon Linux 2/3.1.0
Change
Recent events
Show all

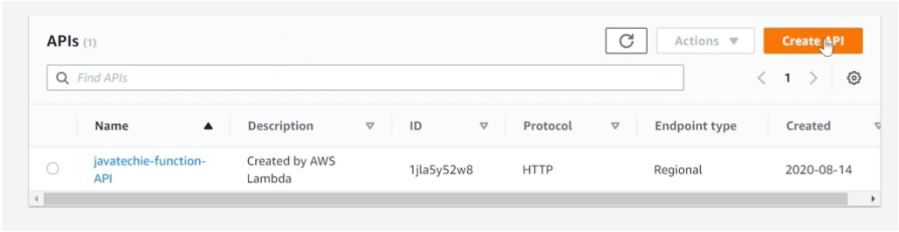

Not secure | javatechie.ap-south-1.elasticbeanstalk.com/book

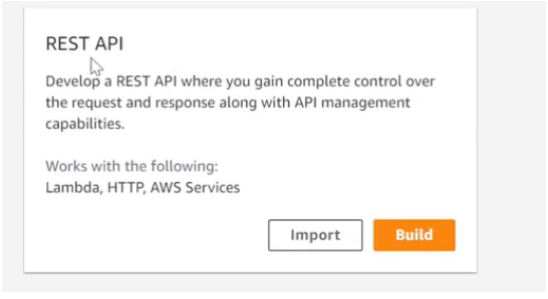

No Environment
POST
http://javatechie.ap-south-1.elasticbeanstalk.com/book
Params
Send
Save
Authorization
Headers (2)
Body
Pre-request Script
Tests
Code
form-data
x-www-form-urlencoded
raw
binary
JSON (application/json)
{
"id":101,
"name":"core java",
"price":5000
}

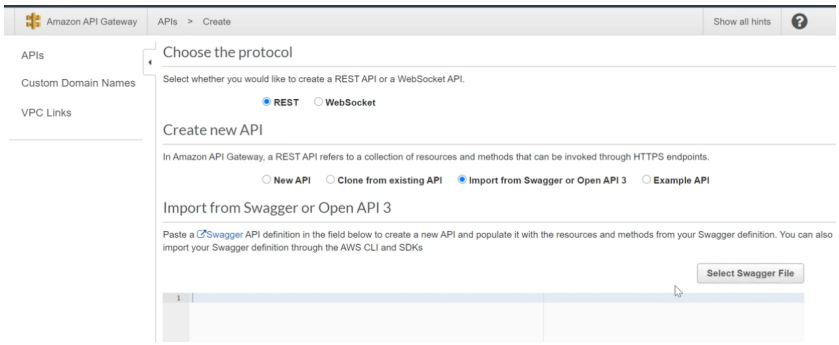

POST
http://javatechie.ap-south-1.elasticbeanstalk.com/book
Params
Send
Save
Authorization
Headers (2)
Body
Pre-request Script
Tests
Code
form-data
x-www-form-urlencoded
raw
binary
JSON (application/json)
{
"id":102,
"name":"spring",
"price":10000
}
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 124 ms
Pretty
Raw
Preview
JSON
{
"id": 102,
"name": "spring",
"price": 10000
}

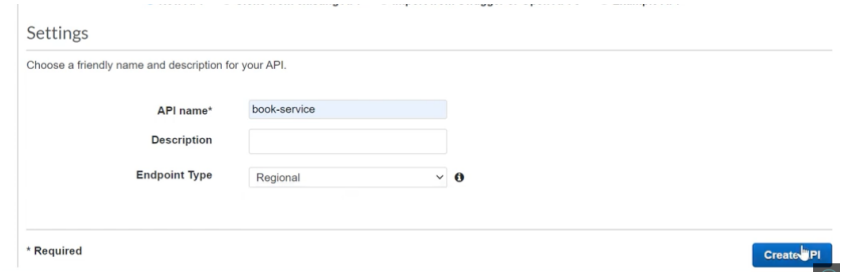

Not secure | javatechie.ap-south-1.elasticbeanstalk.com/book
[
{
"id": 101,
"name": "core java",
"price": 5000
},
{
"id": 102,
"name": "spring",
"price": 10000
}
]

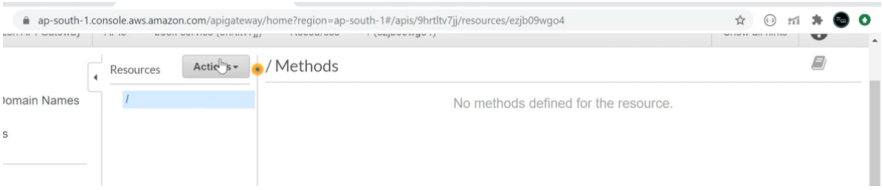

ap-south-1.console.aws.amazon.com/elasticbeanstalk/home?region=ap-south-1#/environment/dashboard?applicationName=book-service&enviro...
aws
Services
Resource Groups
Java Techie
Mumbai
History
Elastic Beanstalk
Console Home
Elastic Container Service
Billing
API Gateway
Lambda
api gatewa
API Gateway
Build, Deploy and Manage APIs
EC2
Lightsail
Lambda
Batch
Elastic Beanstalk
Amazon Managed Blockchain
Satellite
Ground Station
Athena
EMR
CloudSearch
Elasticsearch Service
Kinesis
Alexa for Busi
Amazon Chim
WorkMail
Amazon Hone

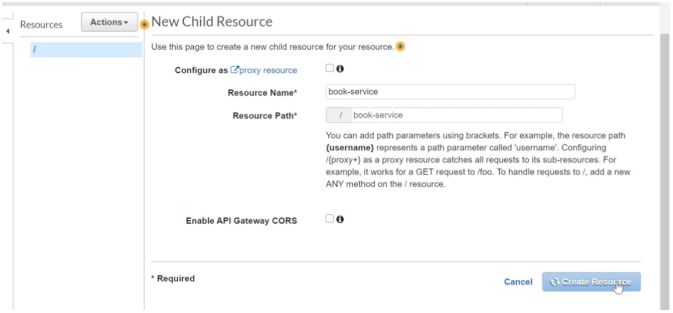

APIs (1)
Actions
Create API
Find APIs
1
Name
Description
ID
Protocol
Endpoint type
Created
javatechie-function-API
Created by AWS Lambda
1jla5y52w8
HTTP
Regional
2020-08-14

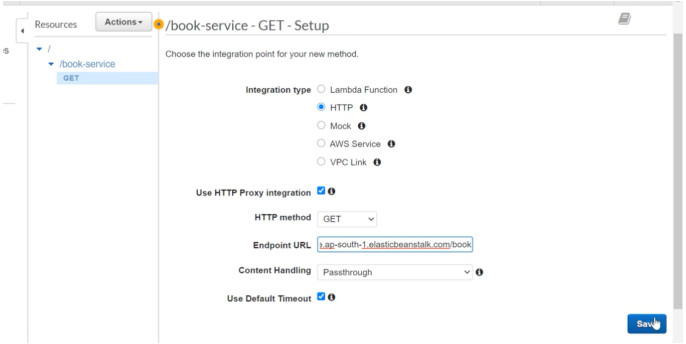

REST API
Develop a REST API where you gain complete control over the request and response along with API management capabilities.
Works with the following:
Lambda, HTTP, AWS Services
Import
Build

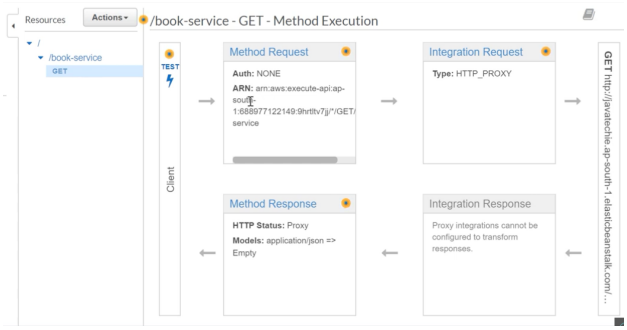

Amazon API Gateway
APIs > Create
Show all hints
APIs
Custom Domain Names
VPC Links
Choose the protocol
Select whether you would like to create a REST API or a WebSocket API.
REST
WebSocket
Create new API
In Amazon API Gateway, a REST API refers to a collection of resources and methods that can be invoked through HTTPS endpoints.
New API
Clone from existing API
Import from Swagger or Open API 3
Example API
Import from Swagger or Open API 3
Paste a Swagger API definition in the field below to create a new API and populate it with the resources and methods from your Swagger definition. You can also import your Swagger definition through the AWS CLI and SDKs
Select Swagger File

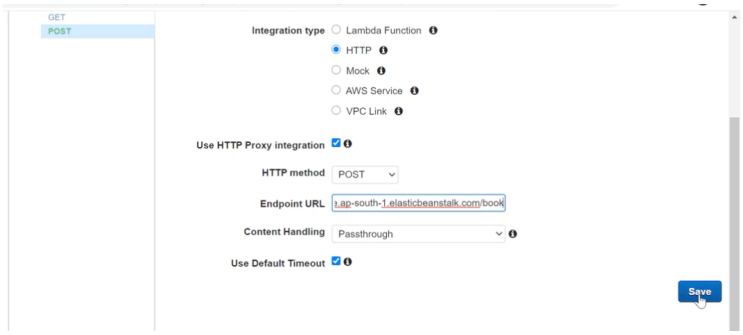

Settings
Choose a friendly name and description for your API.
API name*
book-service
Description
Endpoint Type
Regional
* Required
Create API

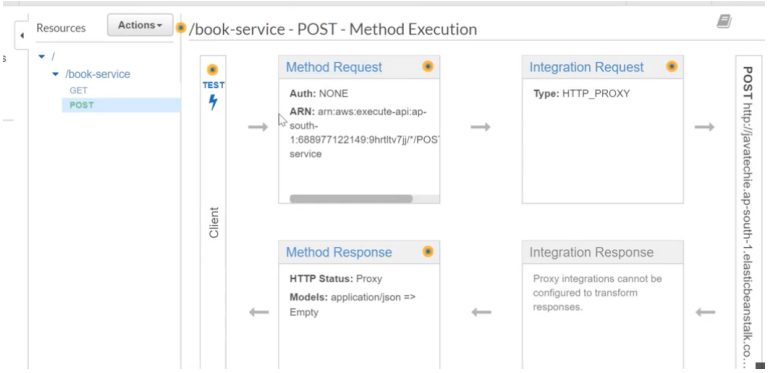

ap-south-1.console.aws.amazon.com/apigateway/home?region=ap-south-1#/apis/9hrtltv7jj/resources/ezjb09wgo4
Resources
Actions
/
/ Methods
No methods defined for the resource.


Resources
Actions
/
New Child Resource
Use this page to create a new child resource for your resource.
Configure as proxy resource
Resource Name*
book-service
Resource Path*
/
book-service
You can add path parameters using brackets. For example, the resource path {username} represents a path parameter called 'username'. Configuring /{proxy+} as a proxy resource catches all requests to its sub-resources. For example, it works for a GET request to /foo. To handle requests to /, add a new ANY method on the / resource.
Enable API Gateway CORS
* Required
Cancel
Create Resource

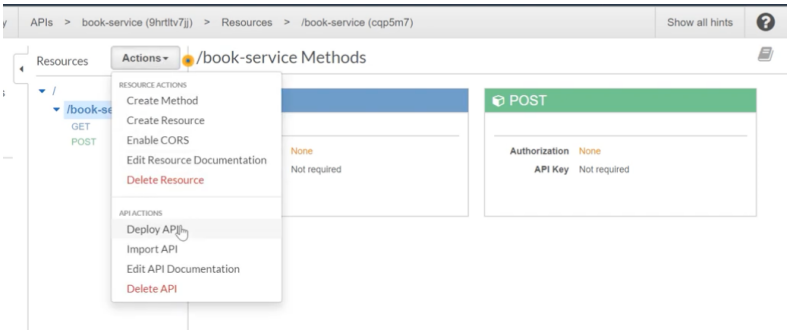

Resources
Actions
/
/book-service
GET
/book-service - GET - Setup
Choose the integration point for your new method.
Integration type
Lambda Function
HTTP
Mock
AWS Service
VPC Link
Use HTTP Proxy integration
HTTP method
GET
Endpoint URL
.ap-south-1.elasticbeanstalk.com/book
Content Handling
Passthrough
Use Default Timeout
Save

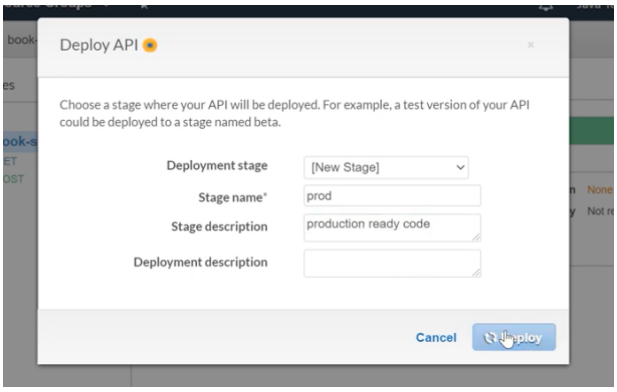

Resources
Actions
/book-service - GET - Method Execution
/book-service
GET
TEST
Client
Method Request
Auth: NONE
ARN: arn:aws:execute-api:ap-south-1:688977122149:9hrtltv7jj/*/GET/service
Integration Request
Type: HTTP_PROXY
GET http://javatechie.ap-south-1.elasticbeanstalk.com/...
Method Response
HTTP Status: Proxy
Models: application/json => Empty
Integration Response
Proxy integrations cannot be configured to transform responses.

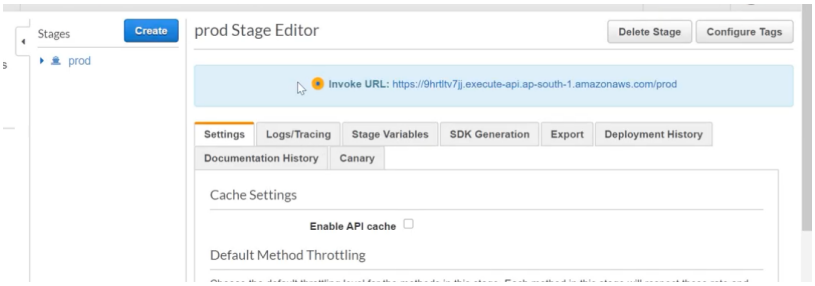

GET
POST
Integration type
Lambda Function
HTTP
Mock
AWS Service
VPC Link
Use HTTP Proxy integration
HTTP method
POST
Endpoint URL
ap-south-1.elasticbeanstalk.com/book
Content Handling
Passthrough
Use Default Timeout
Save

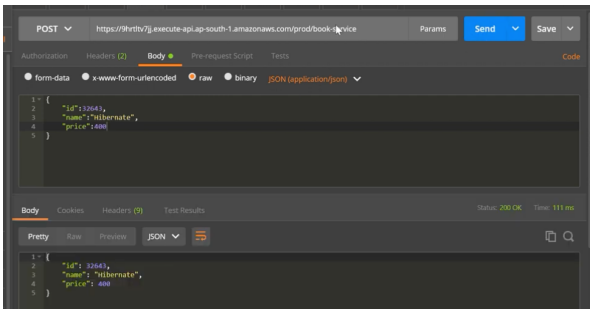

Resources
Actions
/book-service - POST - Method Execution
/book-service
GET
POST
TEST
Client
Method Request
Auth: NONE
ARN: arn:aws:execute-api:ap-south-1:688977122149:9hrtltv7jj/*/POST/service
Integration Request
Type: HTTP_PROXY
POST http://javatechie.ap-south-1.elasticbeanstalk.co...
Method Response
HTTP Status: Proxy
Models: application/json => Empty
Integration Response
Proxy integrations cannot be configured to transform responses.

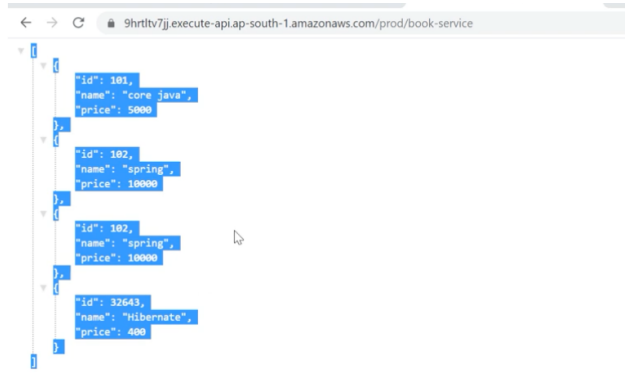
Now deploy


APIs > book-service (9hrtltv7jj) > Resources > /book-service (cqp5m7)
Show all hints
Resources
Actions
/book-service Methods
RESOURCE ACTIONS
Create Method
Create Resource
Enable CORS
Edit Resource Documentation
Delete Resource
API ACTIONS
Deploy API
Import API
Edit API Documentation
Delete API
None
Not required
POST
Authorization None
API Key Not required


Deploy API
Choose a stage where your API will be deployed. For example, a test version of your API could be deployed to a stage named beta.
Deployment stage
[New Stage]
Stage name*
prod
Stage description
production ready code
Deployment description
Cancel
Deploy


Stages
Create
prod
prod Stage Editor
Delete Stage
Configure Tags
Invoke URL: https://9hrtltv7jj.execute-api.ap-south-1.amazonaws.com/prod
Settings
Logs/Tracing
Stage Variables
SDK Generation
Export
Deployment History
Documentation History
Canary
Cache Settings
Enable API cache
Default Method Throttling


POST
https://9hrtltv7jj.execute-api.ap-south-1.amazonaws.com/prod/book-service
Params
Send
Save
Authorization
Headers (2)
Body
Pre-request Script
Tests
Code
form-data
x-www-form-urlencoded
raw
binary
JSON (application/json)
{
"id":32643,
"name":"Hibernate",
"price":400
}
Body
Cookies
Headers (9)
Test Results
Status: 200 OK
Time: 111 ms
Pretty
Raw
Preview
JSON
{
"id": 32643,
"name": "Hibernate",
"price": 400
}


9hrtltv7jj.execute-api.ap-south-1.amazonaws.com/prod/book-service
[
{
"id": 101,
"name": "core java",
"price": 5000
},
{
"id": 102,
"name": "spring",
"price": 10000
},
{
"id": 102,
"name": "spring",
"price": 10000
},
{
"id": 32643,
"name": "Hibernate",
"price": 400
}
]